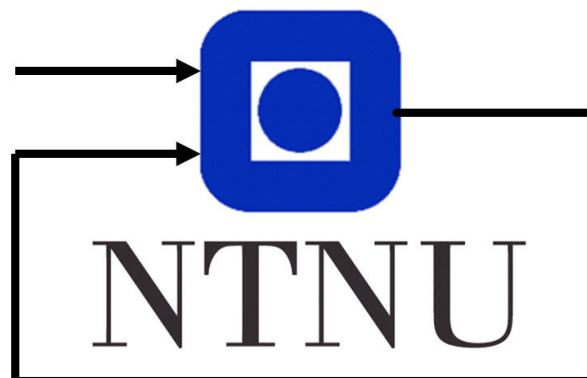


# PROJECT 4: CLASSIFICATION IMAGE

TTT4275 – ESTIMERING, DETEKSJON OG KLASSIFISERING

Student: Kaisa Antonsen  
Student: Jenny Sophie Berntsen

April 17, 2026



Department of Engineering Cybernetics

## Summary

This project is part of the course TTT4275 – Estimation, Detection and Classification, and focuses on gaining practical experience with classification methods. The aim of the project is to implement and evaluate different classifiers, and to understand how theoretical concepts such as linear classifiers, nearest neighbour methods and clustering perform in practice.

The project consisted of two tasks. The first was to classify Iris flowers (*Setosa*, *Versicolor*, *Virginica*) using a linear classifier trained with gradient descent and mean square error (MSE), tested with two different train/test splits and with progressively reduced feature sets. The second task was to classify handwritten digits (0–9) from the MNIST dataset using Nearest Neighbour (NN) and K-Nearest Neighbour (KNN) classifiers, both with and without K-means clustering to reduce the number of templates.

For the Iris task, the linear classifier achieved low error rates (1.67–5.56%) across both splits, with *Setosa* perfectly classified in all experiments. Errors occurred exclusively between *Versicolor* and *Virginica*. Removing features increased the error rate, with performance degrading notably when reduced to one or two features. For MNIST, the 1-NN classifier without clustering achieved a test error rate of 3.09%, while 7-NN gave 4.24%. Applying K-means clustering ( $M = 64$ ) reduced runtime from approximately 11 s to 0.15 s, at the cost of slightly higher error rates of 4.41% and 6.70%, respectively.

Overall, the project demonstrates the trade-off between accuracy and computational efficiency. While using all training data provides the best performance, clustering offers a practical solution for handling large datasets. The project has provided valuable insight into the implementation of classification algorithms, performance evaluation using confusion matrices, and the challenges of real-world data such as variability and class overlap.

# Contents

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                          | <b>1</b>  |
| <b>2</b> | <b>Theory</b>                                | <b>2</b>  |
| 2.1      | Linear classifier . . . . .                  | 2         |
| 2.2      | MSE-Based Training . . . . .                 | 2         |
| 2.3      | Gradient Descent Update Rule . . . . .       | 2         |
| 2.4      | The Template Based Classifier . . . . .      | 3         |
| 2.5      | K-means Clustering . . . . .                 | 4         |
| 2.6      | Performance Metrics . . . . .                | 4         |
| <b>3</b> | <b>The Iris Task</b>                         | <b>6</b>  |
| 3.1      | The Task . . . . .                           | 6         |
| 3.2      | Implementation and Results . . . . .         | 6         |
| <b>4</b> | <b>Classification of Handwritten Numbers</b> | <b>11</b> |
| 4.1      | The Task . . . . .                           | 11        |
| 4.2      | Implementation and Results . . . . .         | 11        |
| <b>5</b> | <b>Conclusion</b>                            | <b>16</b> |
| 5.1      | The Iris Task . . . . .                      | 16        |
| 5.2      | MNIST . . . . .                              | 16        |
|          | <b>References</b>                            | <b>17</b> |
| <b>A</b> | <b>Appendix</b>                              | <b>18</b> |
| A.1      | IRIS code . . . . .                          | 18        |
| A.2      | MNIST code . . . . .                         | 23        |

# 1 Introduction

The goal of this project is to gain practical experience with classification methods within the field of estimation, detection and classification. The project focuses on implementing and evaluating different classifiers, and understanding how theoretical concepts such as linear classifiers, template-based classifiers and clustering translate into practical performance.

Classification is about assigning data to the correct class based on given features. This is used in many real-world applications, such as medical diagnosis, image recognition and pattern analysis [1, p. 1-2]. In this project, we work with two different tasks: classifying Iris flowers based on measurements, and classifying handwritten digits from the MNIST dataset. While these are simplified problems, they represent important types of classification tasks and help illustrate how such methods can be applied in practice. These two tasks are quite different, and help show how classification methods behave for both simple and more complex data.

The report is organized as follows. In Chapter 2, the theoretical background for the classifiers used in this project is presented. Chapter 3 describes the Iris classification task, including implementation and results. In Chapter 4, the classification of handwritten digits using NN and KNN methods, with and without clustering, is presented. Finally, the main findings are summarized and discussed in the conclusion in Chapter 5.

## 2 Theory

### 2.1 Linear classifier

A linear classifier belongs to the category of *discriminant classifiers*, where each class  $\omega$  is described by a discriminant function  $g_i(\mathbf{x})$ . The decision rule assigns an input feature vector  $\mathbf{x}$  to the class with the highest discriminant value:

$$\mathbf{x} \in \omega_j \iff g_j(\mathbf{x}) = \max_i g_i(\mathbf{x}) \quad (1)$$

For the linear classifier, the discriminant function is given by:

$$g_i(\mathbf{x}) = \mathbf{w}_i^T \mathbf{x} + w_{io}, \quad i = 1, \dots, C \quad (2)$$

where  $\mathbf{w}_i$  is a weight vector and  $w_{io}$  is a bias term for class  $\omega_i$  [1, p. 9]. For  $C > 2$  classes, this can be written compactly in matrix form as  $\mathbf{g} = W\mathbf{x} + \mathbf{w}_0$ , where  $W$  is a  $C \times D$  weight matrix,  $D$  is the number of input features, and  $\mathbf{w}_0$  is a bias vector. The bias term  $\mathbf{w}_0$  can be folded into the weight matrix  $W$  by redefining  $[W \ \mathbf{w}_0] \rightarrow W$  and  $[\mathbf{x}^T \ 1]^T \rightarrow \mathbf{x}$ , so that the classifier output reduces to  $\mathbf{g} = W\mathbf{x}$  [1, p. 15].

### 2.2 MSE-Based Training

The parameters of the linear classifier are found by minimizing the mean square error (MSE) between the classifier output and the target values. The MSE is defined as:

$$\text{MSE} = \frac{1}{2} \sum_{k=1}^N (\mathbf{g}_k - \mathbf{t}_k)^T (\mathbf{g}_k - \mathbf{t}_k) \quad (3)$$

where  $\mathbf{t}_k$  is the target vector for sample  $k$ , represented in binary form. Only element  $j$  is 1 if sample  $k$  belongs to class  $\omega_j$ . The classifier output  $\mathbf{g}_k$  is passed through a *sigmoid function* to squash continuous values into the range  $(0, 1)$ :

$$g_{ik} = \text{sigmoid}(z_{ik}) = \frac{1}{1 + e^{-z_{ik}}} \quad (4)$$

where  $\mathbf{z}_k = W\mathbf{x}_k$ . This sigmoid approximates the ideal step function while remaining differentiable, which is necessary for gradient-based training [1, p. 16].

### 2.3 Gradient Descent Update Rule

Since no closed-form solution exists for minimizing the MSE, a gradient descent approach is used. The weight matrix is updated iteratively in the direction opposite to the gradient:

$$W(m) = W(m-1) - \alpha \nabla_W \text{MSE} \quad (5)$$

where  $m$  is the iteration number and  $\alpha$  is the step factor (learning rate). Using the chain rule, the gradient is derived as:

$$\nabla_W \text{MSE} = \sum_{k=1}^N [(\mathbf{g}_k - \mathbf{t}_k) \circ \mathbf{g}_k \circ (1 - \mathbf{g}_k)] \mathbf{x}_k^T \quad (6)$$

where  $\circ$  denotes elementwise multiplication. The step factor  $\alpha$  must be chosen with care: a value too large causes the MSE to oscillate and potentially diverge, while a value too small results in unnecessarily slow convergence [1, p. 17].

## 2.4 The Template Based Classifier

Template based classifiers form a widely used class of classification methods. These classifiers rely directly on comparing an input feature  $\mathbf{x}$  to a set of reference samples, often referred to as templates. Instead of estimating probability distributions, the classification is based on a similarity measure between  $\mathbf{x}$  and the stored templates [1, p. 10].

The simplest template-based classifier is the *Nearest Neighbour (NN)* classifier. In this approach, distance is utilized as the similarity metric by computing the distance between the input  $\mathbf{x}$  and each template. The rule then finds the template which is closest to  $\mathbf{x}$  and assumes that  $\mathbf{x}$  belongs to the same class as this template [1].

Euclidean distance ( $L_2$ -norm) is used as the distance metric and is defined as

$$d(x, \text{ref}_{ik}) = (x - \mu_{ik})^T (x - \mu_{ik}) \quad (7)$$

and the decision rule is then

$$\mathbf{x} \in \omega_j \quad \text{if} \quad d(\mathbf{x}, \boldsymbol{\mu}_j) = \min_i d(\mathbf{x}, \boldsymbol{\mu}_i) \quad (8)$$

That is, the input is assigned to the class of the closest training sample.

The *K-Nearest Neighbor (KNN)* is an extension of the *NN*-classifier in which the classification of an input vector  $\mathbf{x}$  is determined by the majority class among its  $K$  nearest neighbors rather than relying on a single nearest neighbor. This approach improves robustness by reducing the influence of noise and outliers, since the decision is based on a local neighborhood instead of an individual data point [2]. The method involves computing the distance between  $\mathbf{x}$  and all training samples, selecting the  $K$  smallest distances, and assigning the class label based on a majority vote among these neighbors. In cases of ties, the class of the closest sample is typically chosen. The choice of  $K$  is critical, where a small  $K$  leads to low bias but high variance, making the classifier sensitive to noise, whereas a larger  $K$  produces smoother decision boundaries but may blur distinctions between classes.

The most commonly used distance metric is the Euclidean distance due to its simplicity and intuitive geometric interpretation. However, other distance measures can also be used, such as Mahalanobis distance or Manhattan distance.

The NN classifier assigns a class based on the closest training sample, while KNN improves robustness by considering multiple neighbors and using majority voting. Both methods are non-parametric and rely heavily on the choice of distance measure, the value of  $K$ , and the quality of the training data.

## 2.5 K-means Clustering

A major limitation of NN and KNN is computational complexity. For large datasets, computing distances to all training samples becomes expensive. A common approach to reduce complexity is to use clustering methods such as K-means, where cluster centroids are used as templates instead of all training samples.

Clustering is a form of unsupervised learning whose goal is to discover natural groupings among unlabelled data points. Because no ground-truth labels are provided, the algorithm must rely on typically a distance- or density-based objective to partition the data [2]. Unsupervised training arises whenever “we just suspect that some data can be organised into a finite number of distinct groups”, also even when the optimal number of groups is unknown and needs to be estimated as part of the analysis [1].

Beyond grouping data for exploratory analysis, clustering serves an important role as a training procedure for template-based classifiers. A set of class references (templates) can be obtained by running a clustering algorithm on the training samples of each class separately, thereby replacing the full training set with a compact and representative set of references [1].

K-means is the most widely used centroid-based clustering algorithm. Given a dataset of  $n$  points and a chosen number of clusters  $K$ , the algorithm partitions the data by assigning each point to the cluster whose centroid is nearest, measured by squared Euclidean distance [1]. The iterative procedure proceeds as follows [2]:

1. Choose  $K$  initial centroids  $\{\mu_1, \dots, \mu_K\}$ , often done randomly.
2. Initialise  $K$  cluster centroids
3. Assign each data point to the nearest centroid.
4. Update each centroid as the mean of all points assigned to it:

$$\hat{\mu}_i = \frac{1}{N_i} \sum_{k=1}^{N_i} \mathbf{x}_{ik}.$$

5. Repeat steps 3–4 until the cluster assignments converges

A well-known weakness of random initialisation is that the probability of placing all  $K$  centroids in distinct true clusters decreases exponentially with  $K$ . Under the simplifying assumption that every true cluster contains an equal number of points, this probability is approximately  $e^{-K}$  by Stirling’s approximation [2]. The modern remedy is *k-means++*, which places the first centroid randomly and then selects each subsequent centroid with probability proportional to the squared distance from the nearest already-placed centroid. It can be shown that this scheme produces an expected solution that is suboptimal by at most a factor of  $\log k$  [2].

## 2.6 Performance Metrics

To evaluate the classifier, the *error rate* is estimated by counting the number of incorrectly classified samples  $E$  and dividing it by the total number of samples  $N$ :

$$\text{EER} = \frac{E}{N} \tag{9}$$

This is computed separately for the training set ( $\text{EER}_D = E_D/N_D$ ) and the test set ( $\text{EER}_T = E_T/N_T$ ) [1, p. 26]. The *confusion matrix* provides a more detailed view, showing how many samples from each true class are assigned to each predicted class [1, p. 31]. This reveals which classes are most easily confused with one another. A classifier is said to *generalize* well when the difference  $|\text{EER}_T - \text{EER}_D|$  is small, typically in the range of 2–4% [1, p. 29].

## 3 The Iris Task

### 3.1 The Task

The goal with the Iris task is to classify different types of Iris flowers based on measured features. The dataset consists of three classes: Setosa, Versicolor and Virginica, with 50 samples in each class. For each sample, four features are given: sepal length, sepal width, petal length and petal width. Since these features describe physical properties of the flowers, they can be used to separate the classes. The dataset is known to be close to linearly separable, making it suitable for testing a linear classifier.

The task is divided into two parts. In the first part, a linear classifier is trained and tested using two different train/test splits. First, the first 30 samples from each class are used for training and the remaining 20 for testing. Then this is reversed, using the last 30 samples for training and the first 20 for testing. The classifier is trained using a gradient-based method with MSE, where the step size is tuned for convergence. Performance is evaluated using confusion matrices and error rates.

The second part investigates the importance of the features. Histograms are used to study class overlap, and features are gradually removed. The classifier is then retrained using three, two, and one feature(s), and the results are compared.

### 3.2 Implementation and Results

#### Implementation

The Iris dataset is represented as a feature matrix  $X$  and a corresponding label vector  $y$ . The class labels are converted to one-hot encoded target vectors  $\mathbf{t}_k$ , as required by the MSE formulation in Eq. 3. A bias term is included by augmenting each input vector, allowing the classifier to be written on the compact form  $\mathbf{g} = W\mathbf{x}$ , as described in Section 2.1.

The classifier is implemented according to Eq. 2, where the output is computed as  $\mathbf{z}_k = W\mathbf{x}_k$  and passed through the sigmoid function in Eq. 4 to obtain  $\mathbf{g}_k$ . The predicted class is then given by the maximum element of  $\mathbf{g}_k$ , in accordance with the decision rule in Eq. 1.

The parameters  $W$  are found by minimizing the MSE in Eq. 3 using gradient descent. The update rule in Eq. 5, with gradient given in Eq. 6, is implemented in batch form using all training samples in each iteration. The weight matrix is initialized with small random values, and for each choice of step factor  $\alpha$  the training is run for at most 1000 iterations. Training may stop earlier if the change in MSE between two consecutive iterations becomes smaller than a chosen tolerance. The step factor  $\alpha$  is selected by testing several values. The full implementation can be seen in Appendix A.1.

#### Results: Training and Generalization

The step factor  $\alpha$  was tuned experimentally by evaluating the training MSE for different values, as described in Section 2.2. The results are shown in Fig. 1.

For large values of  $\alpha$  (e.g.  $\alpha = 0.1$ ), the MSE does not converge properly. For very small values (e.g.  $\alpha = 0.0001$  and  $\alpha = 10^{-5}$ ), the convergence is stable but slow.

The value  $\alpha = 0.01$  shows some oscillatory behaviour, but still converges to a low MSE within a reasonable number of iterations. This value was therefore used in the remaining experiments.

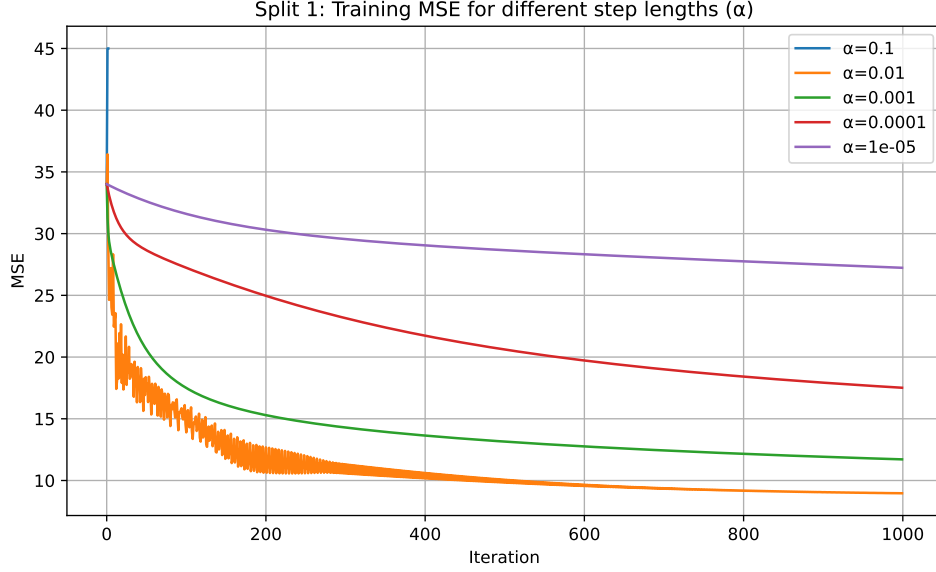


Figure 1: MSE for different step lengths

The confusion matrices and corresponding error rates for both training and test sets are shown in Fig. 2. The results for the two splits are summarized in Table 1. In both cases, Setosa is perfectly classified, which is expected as it is well separated from the other two classes. The misclassifications occur exclusively between Versicolor and Virginica, which are known to overlap in feature space.

| Split                                  | EER <sub>D</sub> | EER <sub>T</sub> |
|----------------------------------------|------------------|------------------|
| (1) Training first 30, testing last 20 | 3.33%            | 5.00%            |
| (2) Training last 30, testing first 20 | 5.56%            | 1.67%            |

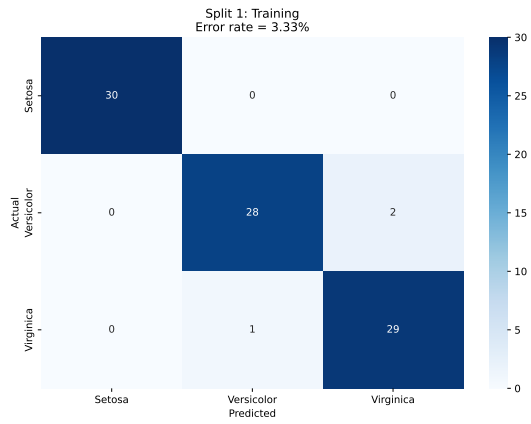
Table 1: Error rates for the two train/test splits

Comparing the two splits, the overall performance is similar, with low error rates in both cases. Split 1 gives a lower training error but a slightly higher test error, while Split 2 shows the opposite. The lower test error in Split 2 suggests that the first 20 samples are slightly easier to classify, likely representing more typical samples of each class, while the last 20 samples used in Split 1 appear somewhat more challenging.

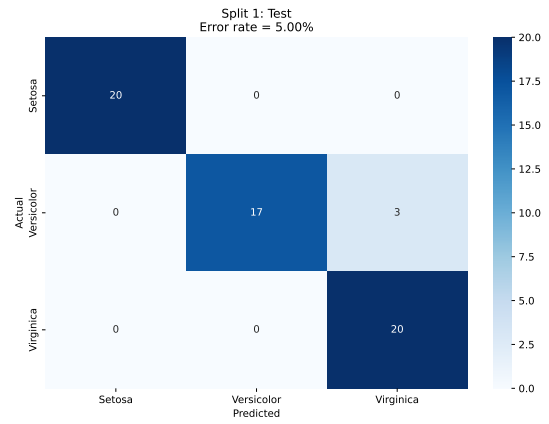
Neither result shows a large gap between  $EER_D$  and  $EER_T$ , which indicates that the classifier generalizes well in both cases. The variation in performance between the splits is therefore most likely due to the limited size and ordering of the dataset.

### Results: Features and Linear Separability

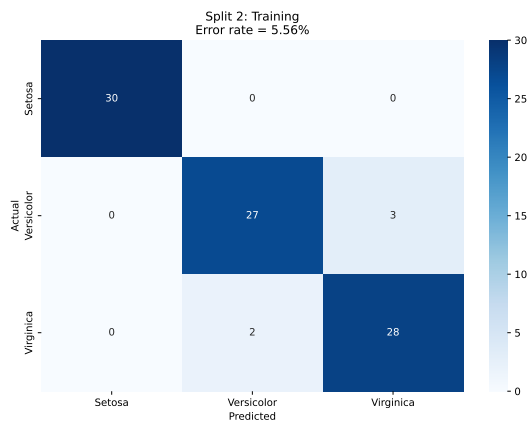
Fig. 3 shows histogram for each of the four features across all three classes. Based on the histograms, **sepal width** shows the most overlap between the three classes and was therefore removed first. The remaining features were then removed in the order of decreasing overlap: sepal length was removed second, and petal length was removed third, leaving only petal width in the single-feature experiment.



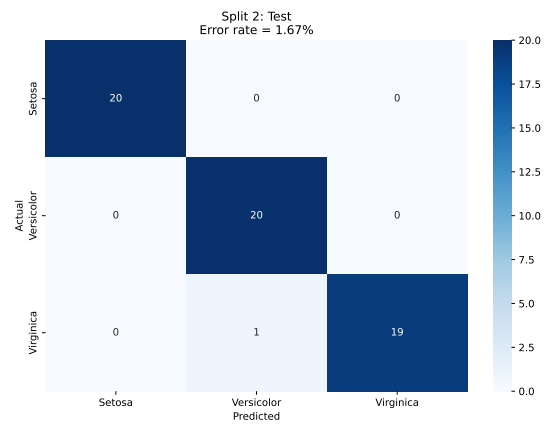
(a) Split 1: Training



(b) Split 1: Test



(c) Split 2: Training



(d) Split 2: Test

Figure 2: Confusion matrices for both train/test splits

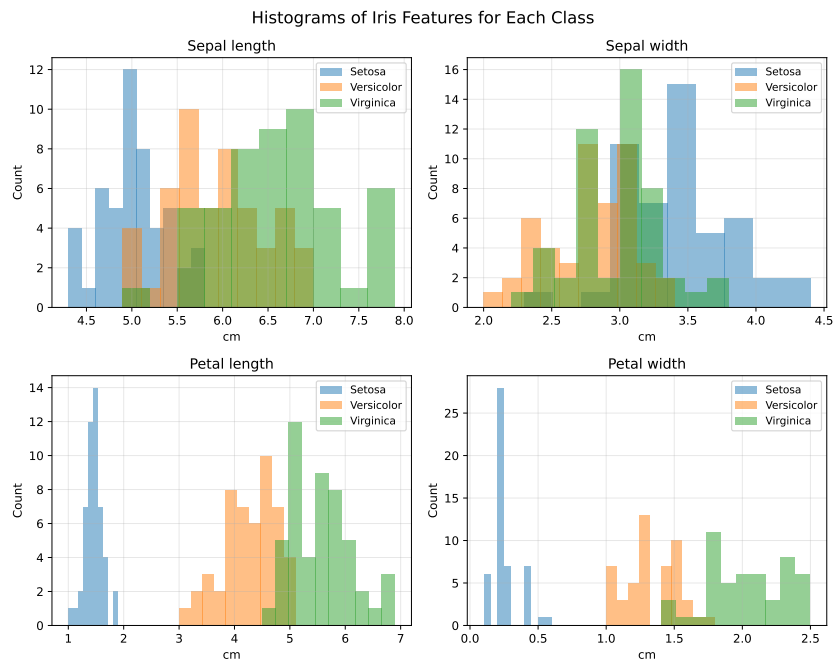


Figure 3: Histograms of Iris features

The classifier was trained and tested using  $\alpha = 0.01$  and Split 1 for all experiments. The results are summarized in Table 2, and the confusion matrices are shown in Fig. 4. As the number of features is reduced, the error rate generally increases, indicating reduced separability between the classes.

The classifier was trained and tested using  $\alpha = 0.01$  and Split 1 for all four experiments. The results are shown in Table 2, and the confusion matrices for the test sets are given in Figs. 4a–4f.

| Feature set                             | EER <sub>D</sub> | EER <sub>T</sub> |
|-----------------------------------------|------------------|------------------|
| All features                            | 3.33%            | 5.00%            |
| 3 features (sepal width removed)        | 3.33%            | 5.00%            |
| 2 features (petal length + petal width) | 6.67%            | 6.67%            |
| 1 feature (petal width only)            | 12.22%           | 8.33%            |

Table 2: Error rates as features are progressively removed

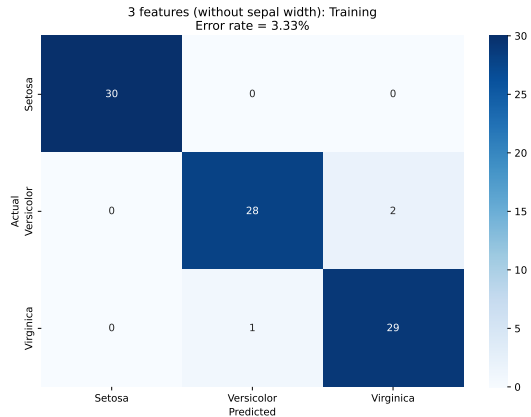
Using three features gives performance similar to the full feature case, with the same error rates for both training and test sets. Only a few misclassifications occur, mainly between Versicolor and Virginica. This suggests that the removed feature (sepal width) has limited importance for linear separability.

When reducing to two features, the error rate increases slightly. The confusion between Versicolor and Virginica becomes more pronounced, indicating that the removed features contribute to separating these two classes.

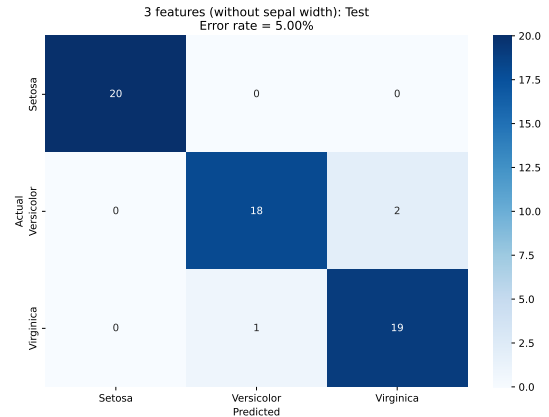
With only one feature (petal width), the error rate increases further, especially for the training set. A significant number of Versicolor samples are misclassified as Virginica, showing that a single feature is not sufficient to separate these classes linearly.

In all cases, Setosa is perfectly classified. This indicates that Setosa is linearly separable from the other two classes even with very limited feature information. In contrast, Versicolor and Virginica show overlap in feature space and are therefore more difficult to separate using a linear classifier.

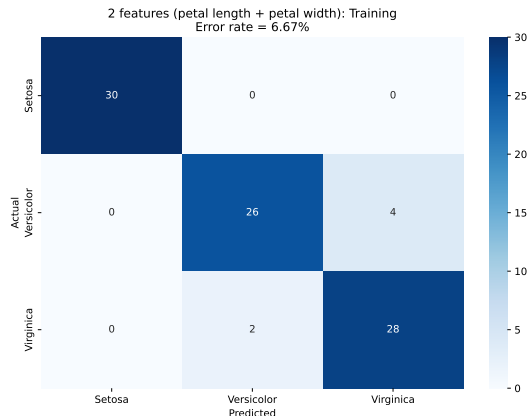
Overall, the results show that while a reduced number of features can still give reasonable performance, multiple features are required to achieve good linear separability, particularly for distinguishing between Versicolor and Virginica. This is consistent with the histogram analysis, where significant overlap between Versicolor and Virginica was observed.



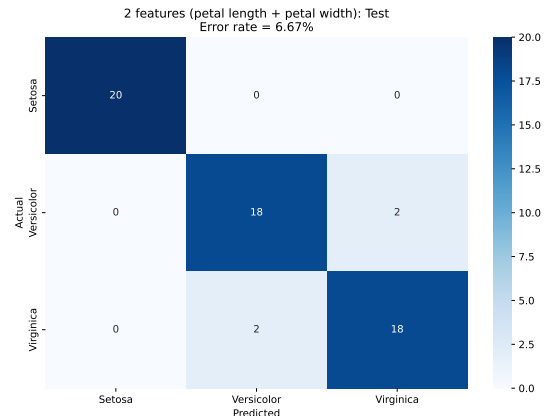
(a) 3 features: Training



(b) 3 features: Test



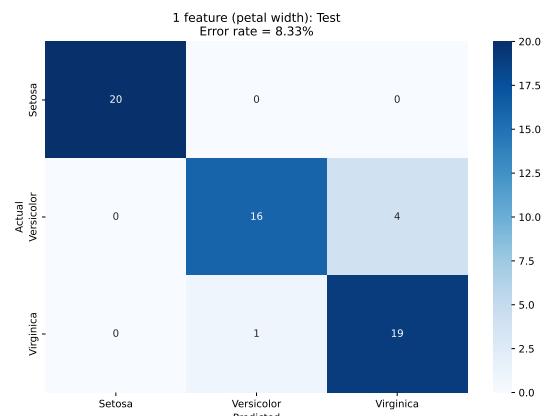
(c) 2 features: Training



(d) 2 features: Test



(e) 1 feature: Training



(f) 1 feature: Test

Figure 4: Confusion matrices for different numbers of features

## 4 Classification of Handwritten Numbers

### 4.1 The Task

In this project, the objective is to develop a classifier capable of recognizing handwritten digits in the range 0–9. The dataset used is the MNIST database, which consists of grayscale images of size  $28 \times 28$  pixels. Each image represents a handwritten digit and is stored as an 8-bit intensity map. The dataset contains a total of 70,000 labeled samples, where 60,000 images are used for training and 10,000 images are reserved for testing.

In this work, *Nearest Neighbor* (NN) and *K-Nearest Neighbor* (KNN) classifiers are implemented using the Euclidean distance as a similarity measure. The task involves classifying each test image by comparing it to the training data and assigning it to the most appropriate class. Furthermore, the impact of different values of  $K$  in the KNN algorithm is investigated, along with considerations of computational cost and classification performance.

The performance of the classifiers is evaluated using metrics such as error rate and confusion matrices, providing insight into how well the methods are able to distinguish between different handwritten digits.

In addition to the NN and KNN classifiers, clustering was applied to reduce the computational complexity of the classification task. For each digit class, K-means clustering is used to partition the training data into  $M = 64$  clusters. The resulting cluster centroids are then used as representative templates instead of all original training samples. These centroids serve as new templates, significantly reducing the number of distance computations required during classification. This approach provides a trade-off between computational efficiency and classification accuracy. By using a reduced set of templates, the classification process becomes faster and less memory-intensive, while still capturing the overall structure of the data. The effect of using clustered templates was evaluated by comparing the performance of the classifier before and after clustering, using metrics such as error rate and confusion matrices.

### 4.2 Implementation and Results

This chapter describes the implementation of the classification methods used in this project. The main objective is to classify handwritten digits from the MNIST dataset using NN and KNN classifiers. Additionally, K-means clustering is applied to reduce the number of templates and improve computational efficiency. The implementation in this project is carried out in Python.

**Part 1** focuses on implementing the NN-based classifier using Euclidean distance. The images have been preprocessed prior to the task, such that they are centered and scaled in preparation for classification. The classification is then performed by computing the Euclidean distance between test samples and training samples with. To prevent overly large distance matrices and reduce computation time, the computations are performed on batches of 1000 images from the test set and 1000 images from the train set.

The nearest-neighbour classifier was implemented using a vectorised approach to compute squared Euclidean distances. Rather than using a prebuilt distance function such as `cdist`, distances were computed block-by-block (1000 test images at a time) using the identity

$$\|\mathbf{p} - \mathbf{q}\|^2 = \|\mathbf{p}\|^2 + \|\mathbf{q}\|^2 - 2\mathbf{p}^\top \mathbf{q},$$

where the matrix product  $\mathbf{QT}^\top$  was computed using Numpy's matrix multiplication. This approach avoids materializing the full  $10\,000 \times 60\,000$  distance matrix in memory at once and is the expanded form of Eq. 7.

The  $K$  nearest neighbours were identified using `np.argsort`, which finds the  $K$  smallest values without performing a full sort, thereby reducing computational cost. For  $K = 1$ , the label of the single nearest template is assigned directly. For  $K > 1$ , the predicted class is determined by a majority vote among the labels of the  $K$  nearest neighbours.

The confusion matrix and error rate was found for both  $K = 1$  and  $K = 7$  in order to compare both scenarios before and after clustering. Displayed in Fig. 5, the error rate for the 1-NN classifier is 3.09%, slightly smaller than the 7-NN classifier with an 4.24% error rate.

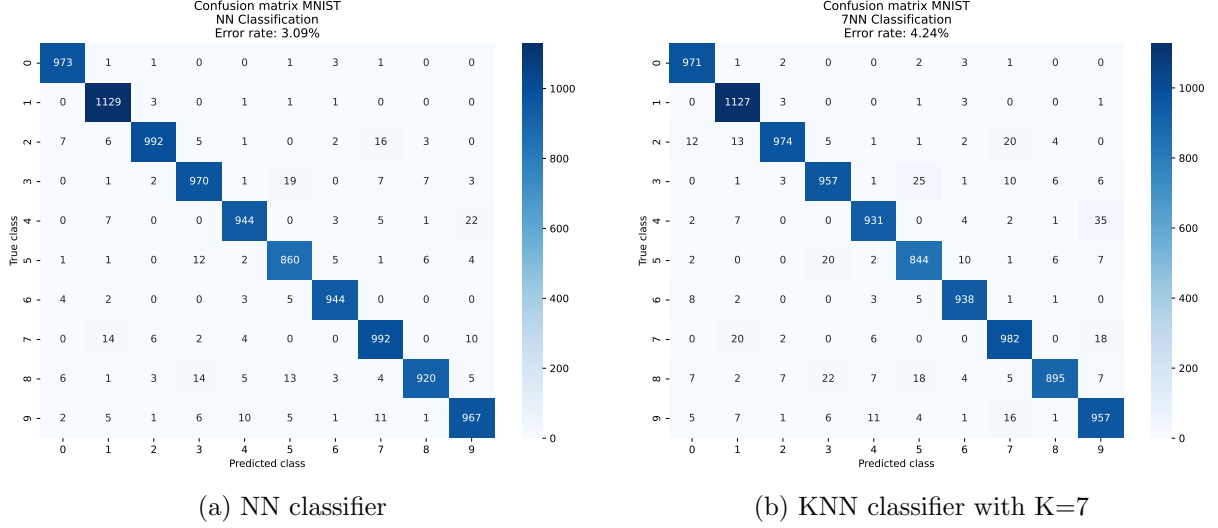


Figure 5: Confusion matrix for KNN classifier without clustering

| Clustering | K | EER <sub>T</sub> | Total runtime (s) |
|------------|---|------------------|-------------------|
| NO         | 1 | 3.09%            | 11.30             |
| NO         | 7 | 4.24%            | 13.30             |
| YES        | 1 | 4.41%            | 0.15              |
| YES        | 7 | 6.70%            | 0.17              |

Table 3: Error rates and total runtime before and after clustering

### Misclassified and Correctly Classified Images

A qualitative analysis of correctly and misclassified images provides insight into the strengths and limitations of the NN classifier. As shown in Fig. 6, correctly classified digits are generally well-centered, clearly written, and exhibit standard shapes. For example, digits such as 1, 4, 5, and 9 are correctly identified when their structure is distinct and consistent with typical representations. These samples have strong similarity to training examples, making them easier to classify using Euclidean distance.

In contrast, the misclassified images displayed in Fig. 7 reveal several common sources of error. Many of these digits are either poorly written, ambiguous, or resemble other classes. For instance, some digits labeled as 3 are classified as 7, and digits labeled as 4 are classified as 9. This can be explained by visual similarities in handwritten forms, such as incomplete loops or unusual stroke patterns. Additionally, some digits appear noisy or distorted, which negatively affects the distance-based comparison.

Another notable observation is that certain digits, such as 0, are occasionally misclassified as 6 or 1. This suggests that the classifier is sensitive to small variations in shape, especially when

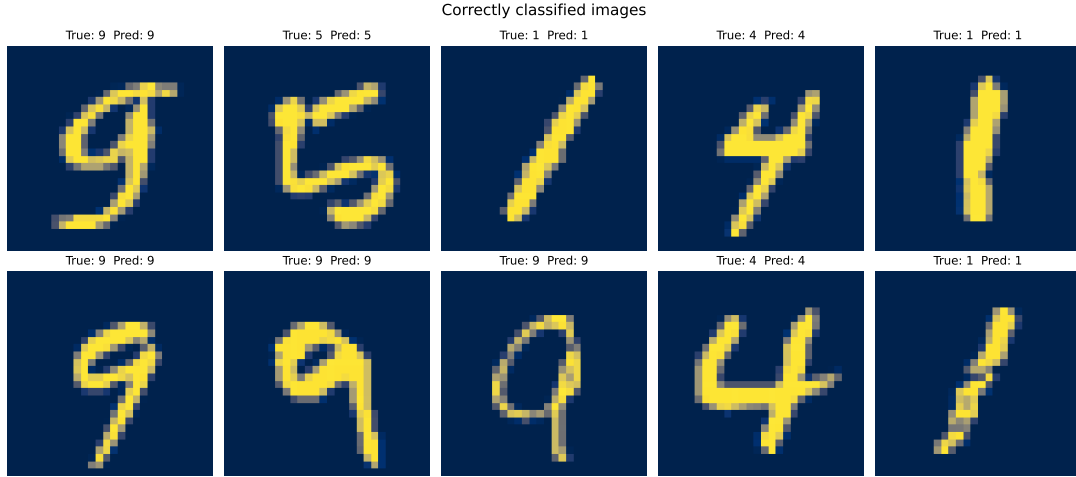


Figure 6: Some images of randomly selected correctly classified images from the NN classifier

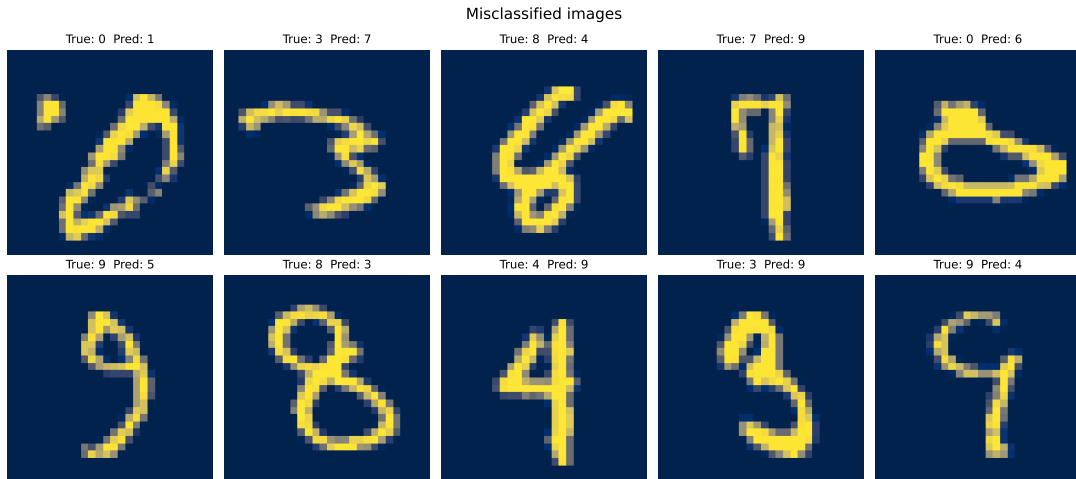


Figure 7: Some images of randomly selected misclassified images from the NN classifier

the distinguishing features are subtle. Since the NN (and KNN) classifier relies directly on raw pixel values without extracting higher-level features, it struggles when different classes have overlapping visual characteristics.

Overall, the correctly classified samples demonstrate that the classifier performs well when the input closely matches typical training examples. However, the misclassified samples highlight the limitations of using Euclidean distance in high-dimensional pixel space, particularly for ambiguous or irregular handwritten digits.

### Clustering

In **part 2** the goal is to reduce the computational cost of the NN classifier by applying K-means clustering to the training data. Instead of using all training samples as templates, each digit class was partitioned into  $M = 64$  clusters, and the corresponding cluster centroids were used as representative templates. The clustered 1-NN classifier outperforms the clustered 7-NN classifier with higher accuracy and slightly shorter computation time, shown in Table. 3. Compared to the 1-NN and 7-NN classifier before clustering, the total runtime is strictly decreased with more than 11s, but the error rate is slightly higher on both clustered classifiers.

The results show that clustering provides a substantial improvement in computational efficiency, as the number of comparisons is greatly reduced. However, this comes at the cost of a slight

decrease in classification accuracy, shown in Fig. 8. This reduction in performance can be explained by the fact that cluster centroids are approximations of the original data and may not capture all variations present in handwritten digits. As a result, fine-grained differences between similar digits may be lost. Despite this, the clustered approach still achieves satisfactory performance, demonstrating that K-means clustering offers a good trade-off between efficiency and accuracy for large datasets such as MNIST.

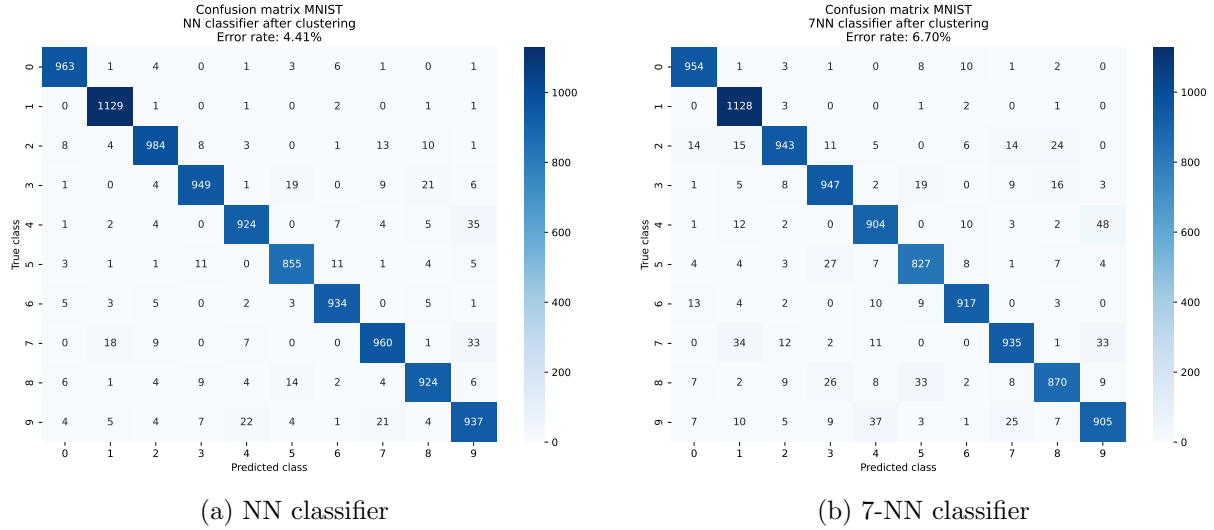


Figure 8: Confusion matrices for K-means MNIST classification

The implementation relies on several important functions and libraries:

- **numpy**: Used for matrix operations and data handling.
- **matplotlib**: Used for visualization of results.
- **sklearn.cluster.KMeans**: Used to perform K-means clustering.
- Custom function **NNClassifier**: Implements both NN and KNN classification.

These functions correspond to common Matlab operations such as distance computations, matrix manipulations, and clustering algorithms.

The full implementation of the classifiers, NN and KNN classification, and clustering, is provided in the Appendix A.2.

## Results

The performance of the classifiers is evaluated using confusion matrices and error rates. The confusion matrix provides a detailed view of how often each digit is correctly classified or misclassified as another digit.

The NN classifier using the full dataset achieves high accuracy but at the cost of significant computational time. When clustering is applied, the number of templates is reduced, resulting in faster classification with a slight decrease in accuracy.

The KNN classifier generally improves robustness compared to NN, especially for ambiguous digits, by considering multiple neighbors in the decision process.

## Discussion

The results show a clear trade-off between accuracy and computational efficiency. Using all training samples in the NN classifier provides the highest accuracy, but it is computationally expensive. Introducing K-means clustering significantly reduces runtime, as the number of templates per class is reduced to  $M = 64$ , but this comes at the cost of some loss in classification performance.

The KNN classifier helps mitigate this loss by considering multiple neighbors, which makes the classifier less sensitive to noise and outliers. However, increasing  $K$  also increases computational cost.

Overall, the combination of clustering and KNN provides a good balance between efficiency and performance, making it a practical solution for large datasets such as MNIST.

## 5 Conclusion

### 5.1 The Iris Task

In this project, a linear classifier was implemented and applied to the Iris dataset based on the theory of discriminant functions and MSE-based training. The implementation followed directly from the theoretical formulation, using one-hot targets, sigmoid activation and gradient descent.

The classifier achieves low error rates for both training and test sets, and the small difference between  $EER_D$  and  $EER_T$  indicates good generalization. The results are similar for both data splits, with only minor variations due to the limited dataset size.

Setosa is perfectly classified in all experiments, while errors occur only between Versicolor and Virginica. The feature analysis shows that removing features increases the error rate, especially when reducing to two or one feature, confirming that multiple features are needed to separate the overlapping classes.

The task provides a clear connection between theory and implementation, and gives useful insight into how linear classifiers behave in practice. It also highlights limitations such as sensitivity to feature choice and dataset size. Overall, the project has given a better understanding of gradient-based training and classification performance evaluation.

### 5.2 MNIST

This project has investigated the use of NN and KNN classifiers for handwritten digit recognition using the MNIST dataset. The results show that the 1-NN classifier outperforms the 7-NN classifier both with and without clustering. On the other hand, the computational time significantly decreased on the clustered data, both with  $K = 1$  and  $K = 7$ .

The analysis of the results shows that correctly classified digits are typically clear and well-formed, while misclassifications often occur between visually similar digits or poorly written samples. This highlights a limitation of distance-based methods, as they rely on pixel-level similarity and do not capture more advanced features.

Overall, the project demonstrates the trade-off between accuracy and efficiency. While NN provides strong performance, it is computationally expensive, and clustering offers a practical compromise. The project has provided valuable experience in implementing classification algorithms, evaluating performance, and understanding the challenges associated with real-world data.

## References

- [1] Magne H. Johnsen. *Classification - Compendium*. 2017.
- [2] Inga Strümke. *Forelesningsnotater Introduksjon til Maskinl ring – TDT4172*. 2025.

# A Appendix

## A.1 IRIS code

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import seaborn as sn
4 from sklearn.metrics import confusion_matrix
5
6
7 # Loading Iris data from file
8 def load_iris(filepath="iris.data"):
9     X = []
10    y = []
11
12    label_map = {
13        "Iris-setosa": 0,
14        "Iris-versicolor": 1,
15        "Iris-virginica": 2
16    }
17
18    with open(filepath, "r") as f:
19        for line in f:
20            line = line.strip()
21            if not line:
22                continue
23            parts = line.split(",")
24            X.append([float(v) for v in parts[:4]])
25            y.append(label_map[parts[4]])
26
27    return np.array(X, dtype=float), np.array(y, dtype=int)
28
29
30 # One-hot encoding
31 def one_hot(y, num_classes=3):
32     T = np.zeros((len(y), num_classes))
33     T[np.arange(len(y)), y] = 1
34     return T
35
36
37 def add_bias(X):
38     return np.hstack([X, np.ones((X.shape[0], 1))])
39
40
41 def sigmoid(z):
42     z = np.clip(z, -50, 50)
43     return 1.0 / (1.0 + np.exp(-z))
44
45
46 def mse(G, T):
47     return 0.5 * np.sum((G - T) ** 2)
48
49
50 def train_linear_classifier(X_train, y_train, alpha=0.01, max_iter=1000, tol=1e-8):
51     T = one_hot(y_train, num_classes=3)
52
53     Xb = add_bias(X_train)
54     _, D = Xb.shape
55     C = T.shape[1]
56
57     rng = np.random.default_rng(42)
58     W = 0.01 * rng.standard_normal((C, D))
```

```

59
60     mse_history = []
61
62     for it in range(max_iter):
63         Z = Xb @ W.T
64         G = sigmoid(Z)
65
66         current_mse = mse(G, T)
67         mse_history.append(current_mse)
68
69         delta = (G - T) * G * (1.0 - G)
70         grad = delta.T @ Xb
71
72         W_new = W - alpha * grad
73
74         if it > 0 and abs(mse_history[-2] - mse_history[-1]) < tol:
75             W = W_new
76             break
77
78         W = W_new
79
80     return W, mse_history
81
82
83 def predict(W, X):
84     Xb = add_bias(X)
85     G = sigmoid(Xb @ W.T)
86     return np.argmax(G, axis=1)
87
88
89 def error_rate(y_true, y_pred):
90     return np.mean(y_true != y_pred)
91
92
93 def run_alpha_experiment(X_train, y_train, X_test, y_test, alphas):
94     results = {}
95
96     for alpha in alphas:
97         W, mse_hist = train_linear_classifier(X_train, y_train, alpha=alpha)
98
99         y_train_pred = predict(W, X_train)
100        y_test_pred = predict(W, X_test)
101
102        train_cm = confusion_matrix(y_train, y_train_pred)
103        test_cm = confusion_matrix(y_test, y_test_pred)
104
105        results[alpha] = {
106            "W": W,
107            "mse_history": mse_hist,
108            "train_cm": train_cm,
109            "test_cm": test_cm,
110            "train_error": error_rate(y_train, y_train_pred),
111            "test_error": error_rate(y_test, y_test_pred),
112            "iterations": len(mse_hist)
113        }
114
115    return results
116
117
118 def plot_mse_histories(results, case_name=""):
119     plt.figure(figsize=(8, 5))
120     for alpha, res in results.items():
121         plt.plot(res["mse_history"], label={alpha})

```

```

122 plt.xlabel("Iteration")
123 plt.ylabel("MSE")
124 plt.title(f"{case_name}: Training MSE for different step lengths (alpha)")
125 plt.legend()
126 plt.grid(True)
127 plt.tight_layout()
128 plt.savefig(f"{case_name}_mse.pdf")
129 plt.show()
130
131
132 def plot_confusion_matrix(cm, class_names, title, error_rate=None):
133     plt.figure(figsize=(8, 6))
134     sn.heatmap(cm, annot=True, fmt="d", cmap="Blues",
135               xticklabels=class_names, yticklabels=class_names)
136     plt.xlabel("Predicted")
137     plt.ylabel("Actual")
138     if error_rate is None:
139         plt.title(title)
140     else:
141         plt.title(f"{title}\nError rate = {error_rate:.2%}")
142     plt.tight_layout()
143     plt.savefig(f"{title}_cm.pdf")
144     plt.show()
145
146
147 def run_case(X, y, split_function, alphas, case_name):
148     X_train, y_train, X_test, y_test = split_function(X, y)
149     results = run_alpha_experiment(X_train, y_train, X_test, y_test, alphas)
150     plot_mse_histories(results, case_name)
151
152     best_alpha = min(
153         results,
154         key=lambda a: (results[a]["train_error"], results[a]["mse_history"][-1])
155     )
156     best_res = results[best_alpha]
157
158     print(f"\n{case_name}")
159     print(f"Best alpha: {best_alpha}")
160     print(f"Training error rate = {best_res['train_error']:.2%}")
161     print(f"Test error rate      = {best_res['test_error']:.2%}")
162
163     class_names = ["Setosa", "Versicolor", "Virginica"]
164     plot_confusion_matrix(best_res["train_cm"], class_names,
165                           f"{case_name}: Training", best_res["train_error"])
166     plot_confusion_matrix(best_res["test_cm"], class_names,
167                           f"{case_name}: Test", best_res["test_error"])
168
169     return results, best_alpha, best_res

```

Listing 1: Helper functions for the Iris task

```

1 def split_first30_last20(X, y):
2     X_train, y_train = [], []
3     X_test, y_test = [], []
4
5     for c in range(3):
6         Xc = X[y == c]
7         yc = y[y == c]
8
9         X_train.append(Xc[:30])
10        y_train.append(yc[:30])
11
12        X_test.append(Xc[30:])
13        y_test.append(yc[30:])

```

```

14
15     X_train = np.vstack(X_train)
16     y_train = np.concatenate(y_train)
17     X_test = np.vstack(X_test)
18     y_test = np.concatenate(y_test)
19
20     return X_train, y_train, X_test, y_test
21
22
23 X, y = load_iris("iris.data")
24
25 alphas = [0.1, 0.01, 0.001, 0.0001, 0.00001]
26
27 results1, best_alpha1, best_res1 = run_case(
28     X, y,
29     split_first30_last20,
30     alphas,
31     "Split 1"
32 )

```

Listing 2: Split 1: First 30 samples of each class for training, last 20 for testing

```

1 def split_last30_first20(X, y):
2     X_train, y_train = [], []
3     X_test, y_test = [], []
4
5     for c in range(3):
6         Xc = X[y == c]
7         yc = y[y == c]
8
9         X_train.append(Xc[20:])
10        y_train.append(yc[20:])
11
12        X_test.append(Xc[:20])
13        y_test.append(yc[:20])
14
15    X_train = np.vstack(X_train)
16    y_train = np.concatenate(y_train)
17    X_test = np.vstack(X_test)
18    y_test = np.concatenate(y_test)
19
20    return X_train, y_train, X_test, y_test
21
22
23 X, y = load_iris("iris.data")
24
25 alphas = [0.1, 0.01, 0.001, 0.0001, 0.00001]
26
27 results2, best_alpha2, best_res2 = run_case(
28     X, y,
29     split_last30_first20,
30     alphas,
31     "Split 2"
32 )

```

Listing 3: Split 2: Last 30 samples of each class for training, first 20 for testing

```

1 def plot_feature_histograms(X, y):
2     feature_names = ["Sepal length", "Sepal width", "Petal length", "Petal width"]
3     class_names = ["Setosa", "Versicolor", "Virginica"]
4
5     fig, axes = plt.subplots(2, 2, figsize=(10, 8))
6     axes = axes.flatten()

```

```

7
8     for i in range(4):
9         ax = axes[i]
10
11         for c in range(3):
12             ax.hist(
13                 X[y == c, i],
14                 bins=10,
15                 alpha=0.5,
16                 label=class_names[c]
17             )
18
19         ax.set_title(feature_names[i])
20         ax.set_xlabel("cm")
21         ax.set_ylabel("Count")
22         ax.grid(True, alpha=0.3)
23         ax.legend(fontsize=9)
24
25     fig.suptitle("Histograms of Iris Features for Each Class", fontsize=14)
26     plt.tight_layout()
27     plt.savefig("histograms.pdf")
28     plt.show()
29
30
31 X, y = load_iris("iris.data")
32 plot_feature_histograms(X, y)

```

Listing 4: Histograms of Iris features for each class

```

1 # 3 features: sepal length, petal length and petal width
2 X_3 = X[:, [0, 2, 3]]
3
4 results3, best_alpha3, best_res3 = run_case(
5     X_3, y,
6     split_first30_last20,
7     alphas,
8     "3 features (without sepal width)"
9 )

```

Listing 5: Removal of one feature: sepal width

```

1 # 2 features: petal length and petal width
2 X_2 = X[:, [2, 3]]
3
4 results2, best_alpha2, best_res2 = run_case(
5     X_2, y,
6     split_first30_last20,
7     alphas,
8     "2 features (petal length + petal width)"
9 )

```

Listing 6: Removal of two features: sepal width and sepal length

```

1 # 1 feature: petal width
2 X_1 = X[:, [3]]
3
4 results1, best_alpha1, best_res1 = run_case(
5     X_1, y,
6     split_first30_last20,
7     alphas,
8     "1 feature (petal width)"
9 )

```

Listing 7: Removal of three features: sepal width, sepal length and petal length

## A.2 MNIST code

```
1
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import scipy.io
5 import seaborn as sns
6 from scipy.spatial.distance import cdist
7 from sklearn.cluster import KMeans
8 import time
9
10 # Load data from .mat file
11 data = scipy.io.loadmat('data_all.mat')
12 trainv = data['trainv'].astype(np.float32) # (60000, 784)
13 trainlab = data['trainlab'].flatten().astype(int)
14 testv = data['testv'].astype(np.float32) # (10000, 784)
15 testlab = data['testlab'].flatten().astype(int)
16
17 # Generate the confusion matrix
18 def generateConfMatrix(predicted_labels, true_labels):
19     # 0-axis / y-axis: true class
20     # 1-axis / x-axis: predicted class
21     conf_matrix = np.zeros((10,10), dtype=int)
22
23     for i in range(len(predicted_labels)):
24         predicted_class = predicted_labels[i]
25         true_class = true_labels[i]
26
27         conf_matrix[true_class][predicted_class] += 1
28
29     return conf_matrix
30
31 # Calculate the precision, recall and accuracy
32 def calculatePerformance(conf_matrix):
33     # Precision: correctly classified as class i / all classified as class i
34     precision = np.diag(conf_matrix) / np.sum(conf_matrix, axis=0)
35     # Recall: correctly classified as class i / all true class i
36     recall = np.diag(conf_matrix) / np.sum(conf_matrix, axis=1)
37     # Accuracy: correctly classified / all samples
38     accuracy = np.trace(conf_matrix) / np.sum(conf_matrix)
39     return precision, recall, accuracy
40
41 # Generate a plt.Figure representation of the confusion matrix
42 def generateConfMatrixFigure(conf_matrix, title, figtitle = ""):
43     confusionM_fig = plt.figure(figsize = (10,7))
44     confusionM_fig = sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues")
45
46     _, _, accuracy = calculatePerformance(conf_matrix)
47     confusionM_fig.set_title(f"{title}\nError rate: {1-accuracy:.2%}")
48
49     confusionM_fig.set_ylabel("True class")
50     confusionM_fig.set_xlabel("Predicted class")
51
52     confusionM_fig.set_xticklabels(list(range(0, conf_matrix.shape[0])))
53     confusionM_fig.set_yticklabels(list(range(0, conf_matrix.shape[0])))
54
55     plt.savefig(f"figures/confmatrix_" + figtitle + ".pdf", format="pdf",
56               bbox_inches="tight")
57     return confusionM_fig
```

Listing 8: Helperfunctions to generate confusion matrix and calculate performance

```
1
2 def NNClassifier(data_to_be_classified, labeled_data, k):
```

```

3     n_samples = data_to_be_classified.shape[0]
4     block_size = 1000
5     kNN_index = np.zeros((n_samples, k), dtype=int)
6
7     # Precompute squared norms of templates (done once, outside the loop)
8     tt = (labeled_data ** 2).sum(axis=1)
9
10    for i in range(0, n_samples, block_size):
11        q = data_to_be_classified[i:i+block_size]
12        qq = (q ** 2).sum(axis=1, keepdims=True)
13        d2 = qq + tt - 2 * (q @ labeled_data.T) # squared L2
14        kNN_index[i:i+block_size] = np.argpartition(d2, kth=k, axis=1)[:k]
15
16    return kNN_index

```

Listing 9: NN classifier function computing vectorised squared Euclidean distance

```

1
2 t0 = time.perf_counter()
3 kNN_index = NNClassifier(testv, trainv, k=1)
4 t1 = time.perf_counter()
5 print(f"Classified in {t1-t0:.1f}s")
6
7 NN_index = kNN_index[:, 0] # Returns only the index of the nearest neighbor
8 NN_pred = trainlab[NN_index]
9
10
11 conf_matrix1 = generateConfMatrix(NN_pred, testlab)
12 generateConfMatrixFigure(conf_matrix1, "Confusion matrix MNIST\nNN
    Classification", "1a_NN_ConfMatrix")
13 precision, recall, accuracy = calculatePerformance(conf_matrix1)
14 print(f"Accuracy: {accuracy:.2%}")
15 for i in range(10):
16     print(f"{i}: precision: {precision[i]:.2%}, recall: {recall[i]:.2%}")

```

Listing 10: Display confusion matrix and error rates for the NN classifier before clustering

```

1
2 t0 = time.perf_counter()
3 kNN_index = NNClassifier(testv, trainv, k=7)
4 t1 = time.perf_counter()
5 print(f"Classified in {t1-t0:.1f}s")
6
7 NN_index = kNN_index[:, 0]
8 NN_pred = trainlab[NN_index]
9
10 conf_matrix2 = generateConfMatrix(NN_pred, testlab)
11 generateConfMatrixFigure(conf_matrix2, "Confusion matrix MNIST\n7NN
    Classification", "1a_7NN_ConfMatrix")
12 precision, recall, accuracy = calculatePerformance(conf_matrix2)
13 print(f"Accuracy: {accuracy:.2%}")
14 for i in range(10):
15     print(f"{i}: precision: {precision[i]:.2%}, recall: {recall[i]:.2%}")

```

Listing 11: Display confusion matrix and error rates for KNN classifier with K=7 before clustering

```

1
2 wrong_index = np.where(NN_pred != testlab)[0]
3
4 fig, axes = plt.subplots(2, 5, figsize=(13, 6))
5 np.random.seed(42)
6 sample = np.random.choice(wrong_index, size=10, replace=False)
7

```

```

8 for i, idx in enumerate(sample):
9     axes.flat[i].imshow(testv[idx].reshape(28, 28), cmap='cividis')
10    axes.flat[i].set_title(f'True: {testlab[idx]} Pred: {NN_pred[idx]}',
11                           fontsize=10)
12    axes.flat[i].axis('off')
13 plt.suptitle(f'Misclassified images', fontsize=13)
14 plt.tight_layout()
15 plt.savefig("figures/misclassified.pdf", format="pdf", bbox_inches="tight")
16 plt.show()

```

Listing 12: Display misclassified images

```

1
2 correct_index = np.where(NN_pred == testlab)[0]
3
4 fig, axes = plt.subplots(2, 5, figsize=(13, 6))
5 np.random.seed(42)
6 sample = np.random.choice(correct_index, size=10, replace=False)
7
8 for i, idx in enumerate(sample):
9     axes.flat[i].imshow(testv[idx].reshape(28, 28), cmap='cividis')
10    axes.flat[i].set_title(f'True: {testlab[idx]} Pred: {NN_pred[idx]}',
11                           fontsize=10)
12    axes.flat[i].axis('off')
13 plt.suptitle(f'Correctly classified images', fontsize=13)
14 plt.tight_layout()
15 plt.savefig("figures/correctly_classified.pdf", format="pdf", bbox_inches="tight")
16 plt.show()

```

Listing 13: Display correctly classified images

```

1
2 M = 64 # Clusters per class
3 def kmeans_clustering(trainv, trainlab, M):
4     all_templates = []
5     all_template_labels = []
6
7     for digit in range(10):
8         index = np.where(trainlab == digit)[0]
9         train_digit = trainv[index]
10
11         print(f"Clustering class {digit} ({len(index)} samples)...", end=" ",
12               flush=True)
13         km = KMeans(n_clusters=M, random_state=42, n_init=3, max_iter=200)
14         km.fit(train_digit)
15
16         all_templates.append(km.cluster_centers_.astype(np.float32))
17         all_template_labels.extend([digit] * M)
18         print("done")
19
20     all_templates = np.ascontiguousarray(np.vstack(all_templates))
21     all_template_labels = np.array(all_template_labels)
22
23     print(f"\nDone. Template set: {all_templates.shape}")
24     return all_templates, all_template_labels
25 all_templates, all_template_labels = kmeans_clustering(trainv, trainlab, M)

```

Listing 14: Function for clustering using K-means

```

1
2 t0 = time.perf_counter()
3 kNN_index_kmeans = NNClassifier(testv, all_templates, k=1)
4 t1 = time.perf_counter()
5 print(f"Classification time: {t1-t0:.2f}s")
6
7 NN_pred_kmeans = all_template_labels[kNN_index_kmeans[:, 0]]
8
9 conf_matrix_km = generateConfMatrix(NN_pred_kmeans, testlab)
10 generateConfMatrixFigure(conf_matrix_km, "Confusion matrix MNIST\nNN classifier
    after clustering", "2b_kmeans_NN")
11
12 precision, recall, accuracy = calculatePerformance(conf_matrix_km)
13 print(f"Accuracy: {accuracy:.2%}")
14 for i in range(10):
15     print(f"{i}: precision: {precision[i]:.2%}, recall: {recall[i]:.2%}")

```

Listing 15: Display confusion matrix and error rate for the NN classifier after clustering

```

1
2 t0 = time.perf_counter()
3 kNN_index_k7 = NNClassifier(testv, all_templates, k=7)
4 t1 = time.perf_counter()
5 print(f"Classification time: {t1-t0:.2f}s")
6
7 kNN_labels = all_template_labels[kNN_index_k7]
8 KNN_pred_k7 = np.array([np.bincount(row, minlength=10).argmax() for row in
    kNN_labels])
9
10 conf_matrix_k7 = generateConfMatrix(KNN_pred_k7, testlab)
11 generateConfMatrixFigure(conf_matrix_k7, "Confusion matrix MNIST\n7NN classifier
    after clustering", "2c_kmeans_7NN")
12
13 precision, recall, accuracy = calculatePerformance(conf_matrix_k7)
14 print(f"Accuracy: {accuracy:.2%}")
15 for i in range(10):
16     print(f"{i}: precision: {precision[i]:.2%}, recall: {recall[i]:.2%}")

```

Listing 16: Display confusion matrix and error rate for the KNN classifier after clustering with K=7